

Exercise 1: Implementing the Singleton Pattern

Aim

To implement the Singleton Design Pattern in Java and ensure that only one instance of a class is created throughout the application lifecycle.

Objective

The Singleton Pattern restricts the instantiation of a class to a single object and provides a global point of access to that instance. This is useful for classes such as logging utilities, configuration managers, and database connections where only one instance is required.

Algorithm

1. Create a class named Logger.
2. Declare a private static variable of type Logger to store the single instance.
3. Make the constructor of the class private to prevent object creation from outside the class.
4. Create a public static method getInstance() that returns the single instance of the class.
5. If the instance does not exist, create it; otherwise, return the existing instance.
6. Create a test class to verify that multiple calls to getInstance() return the same object.

Program

Logger.java

```
public class Logger {  
  
    // Private static instance  
    private static Logger instance;  
  
    // Private constructor  
    private Logger() {  
        System.out.println("Logger instance created.");  
    }  
  
    // Public method to get the single instance  
    public static Logger getInstance() {  
        if (instance == null) {  
            instance = new Logger();  
        }  
        return instance;  
    }  
  
    // Logging method  
    public void log(String message) {  
        System.out.println("LOG: " + message);  
    }  
}
```

SingletonTest.java

```

public class SingletonTest {

    public static void main(String[] args) {

        Logger logger1 = Logger.getInstance();
        Logger logger2 = Logger.getInstance();

        logger1.log("Application started.");
        logger2.log("User logged in.");

        if (logger1 == logger2) {
            System.out.println("Only one Logger instance exists.");
        } else {
            System.out.println("Multiple Logger instances exist.");
        }
    }
}

```

Output

Logger instance created.
 LOG: Application started.
 LOG: User logged in.
 Only one Logger instance exists.

Result

The Singleton Design Pattern was successfully implemented. The Logger class created only one instance, and all requests for the Logger object returned the same instance, ensuring consistent logging throughout the application.

Exercise 2: Implementing the Factory Method Pattern

Aim

To implement the Factory Method Design Pattern in Java for creating different types of documents such as Word, PDF, and Excel documents.

Objective

The Factory Method Pattern provides an interface for creating objects while allowing subclasses to decide which class to instantiate. This pattern promotes loose coupling and makes the system easier to extend with new document types.

Algorithm

1. Create a document interface named Document.
2. Define a common method such as open() in the interface.
3. Create concrete document classes:
 - WordDocument
 - PdfDocument
 - ExcelDocument
4. Create an abstract factory class DocumentFactory with an abstract method createDocument().

5. Create concrete factory classes:
 - WordDocumentFactory
 - PdfDocumentFactory
 - ExcelDocumentFactory
6. Implement the createDocument() method in each factory class to return the appropriate document object.
7. Create a test class to demonstrate the creation and usage of different document types through the factory classes.

Program

Document.java

```
public interface Document {  
    void open();  
}
```

WordDocument.java

```
public class WordDocument implements Document {  
  
    @Override  
    public void open() {  
        System.out.println("Opening Word Document.");  
    }  
}
```

PdfDocument.java

```
public class PdfDocument implements Document {  
  
    @Override  
    public void open() {  
        System.out.println("Opening PDF Document.");  
    }  
}
```

ExcelDocument.java

```
public class ExcelDocument implements Document {  
  
    @Override  
    public void open() {  
        System.out.println("Opening Excel Document.");  
    }  
}
```

DocumentFactory.java

```
public abstract class DocumentFactory {  
  
    public abstract Document createDocument();  
}
```

WordDocumentFactory.java

```
public class WordDocumentFactory extends DocumentFactory {
```

```

    @Override
    public Document createDocument() {
        return new WordDocument();
    }
}

```

PdfDocumentFactory.java

```

public class PdfDocumentFactory extends DocumentFactory {

    @Override
    public Document createDocument() {
        return new PdfDocument();
    }
}

```

ExcelDocumentFactory.java

```

public class ExcelDocumentFactory extends DocumentFactory {

    @Override
    public Document createDocument() {
        return new ExcelDocument();
    }
}

```

FactoryMethodTest.java

```

public class FactoryMethodTest {

    public static void main(String[] args) {

        DocumentFactory wordFactory = new WordDocumentFactory();
        Document wordDoc = wordFactory.createDocument();
        wordDoc.open();

        DocumentFactory pdfFactory = new PdfDocumentFactory();
        Document pdfDoc = pdfFactory.createDocument();
        pdfDoc.open();

        DocumentFactory excelFactory = new ExcelDocumentFactory();
        Document excelDoc = excelFactory.createDocument();
        excelDoc.open();
    }
}

```

Output

Opening Word Document.
 Opening PDF Document.
 Opening Excel Document.

Result

The Factory Method Design Pattern was successfully implemented. Different document types were created using their respective factory classes, demonstrating object creation through the factory method while maintaining loose coupling and extensibility.

Exercise 3: Implementing the Builder Pattern

Aim

To implement the Builder Design Pattern in Java for constructing complex objects such as a Computer with multiple optional components.

Objective

The Builder Pattern separates the construction of a complex object from its representation, allowing the same construction process to create different configurations of the object. This pattern improves code readability and flexibility when dealing with objects that have many optional parameters.

Algorithm

1. Create a class named Computer with attributes such as CPU, RAM, Storage, and Graphics Card.
2. Make the constructor of the Computer class private.
3. Create a static nested class named Builder inside the Computer class.
4. Add methods in the Builder class to set the values of the attributes.
5. Implement a build() method in the Builder class that returns a Computer object.
6. Pass the Builder object to the private constructor of Computer.
7. Create a test class to construct different computer configurations using the Builder pattern.

Program

Computer.java

```
public class Computer {

    private String cpu;
    private int ram;
    private int storage;
    private String graphicsCard;

    // Private constructor
    private Computer(Builder builder) {
        this.cpu = builder.cpu;
        this.ram = builder.ram;
        this.storage = builder.storage;
        this.graphicsCard = builder.graphicsCard;
    }

    public void displayConfiguration() {
        System.out.println("CPU: " + cpu);
    }
}
```

```

        System.out.println("RAM: " + ram + " GB");
        System.out.println("Storage: " + storage + " GB");
        System.out.println("Graphics Card: " + graphicsCard);
        System.out.println();
    }

    // Static Nested Builder Class
    public static class Builder {

        private String cpu;
        private int ram;
        private int storage;
        private String graphicsCard;

        public Builder setCPU(String cpu) {
            this.cpu = cpu;
            return this;
        }

        public Builder setRAM(int ram) {
            this.ram = ram;
            return this;
        }

        public Builder setStorage(int storage) {
            this.storage = storage;
            return this;
        }

        public Builder setGraphicsCard(String graphicsCard) {
            this.graphicsCard = graphicsCard;
            return this;
        }

        public Computer build() {
            return new Computer(this);
        }
    }
}

```

BuilderPatternTest.java

```

public class BuilderPatternTest {

    public static void main(String[] args) {

```

```

Computer gamingPC = new Computer.Builder()
    .setCPU("Intel Core i9")
    .setRAM(32)
    .setStorage(1000)
    .setGraphicsCard("NVIDIA RTX 4080")
    .build();

Computer officePC = new Computer.Builder()
    .setCPU("Intel Core i5")
    .setRAM(16)
    .setStorage(512)
    .setGraphicsCard("Integrated Graphics")
    .build();

System.out.println("Gaming PC Configuration:");
gamingPC.displayConfiguration();

System.out.println("Office PC Configuration:");
officePC.displayConfiguration();
}
}

```

Output

Gaming PC Configuration:

CPU: Intel Core i9

RAM: 32 GB

Storage: 1000 GB

Graphics Card: NVIDIA RTX 4080

Office PC Configuration:

CPU: Intel Core i5

RAM: 16 GB

Storage: 512 GB

Graphics Card: Integrated Graphics

Result

The Builder Design Pattern was successfully implemented. Different configurations of the Computer object were created using the Builder class, demonstrating a flexible and readable approach to constructing complex objects with multiple optional parameters.

Exercise 4: Implementing the Adapter Pattern

Aim

To implement the Adapter Design Pattern in Java for integrating multiple third-party payment gateways with different interfaces into a unified payment processing system.

Objective

The Adapter Pattern allows incompatible interfaces to work together by converting the interface of one class into another interface expected by the client. This pattern is useful when integrating external systems or third-party libraries with different interfaces.

Algorithm

1. Create a target interface named `PaymentProcessor` with a method `processPayment()`.
2. Create different payment gateway classes with their own specific methods.
3. Create adapter classes for each payment gateway.
4. Implement the `PaymentProcessor` interface in the adapter classes.
5. Translate the calls from `processPayment()` to the corresponding gateway-specific methods.
6. Create a test class to demonstrate payment processing using different gateways through the adapters.

Program

PaymentProcessor.java

```
public interface PaymentProcessor {  
    void processPayment(double amount);  
}
```

PayPalGateway.java

```
public class PayPalGateway {  
  
    public void sendPayment(double amount) {  
        System.out.println("Processing payment of Rs." + amount + " through PayPal.");  
    }  
}
```

StripeGateway.java

```
public class StripeGateway {  
  
    public void makePayment(double amount) {  
        System.out.println("Processing payment of Rs." + amount + " through Stripe.");  
    }  
}
```

PayPalAdapter.java

```
public class PayPalAdapter implements PaymentProcessor {  
  
    private PayPalGateway payPalGateway;  
  
    public PayPalAdapter(PayPalGateway payPalGateway) {  
        this.payPalGateway = payPalGateway;  
    }  
  
    @Override  
    public void processPayment(double amount) {  
        payPalGateway.sendPayment(amount);  
    }  
}
```

```
}
```

StripeAdapter.java

```
public class StripeAdapter implements PaymentProcessor {
```

```
    private StripeGateway stripeGateway;
```

```
    public StripeAdapter(StripeGateway stripeGateway) {  
        this.stripeGateway = stripeGateway;  
    }  
}
```

```
    @Override  
    public void processPayment(double amount) {  
        stripeGateway.makePayment(amount);  
    }  
}
```

AdapterPatternTest.java

```
public class AdapterPatternTest {
```

```
    public static void main(String[] args) {
```

```
        PaymentProcessor payPalProcessor =  
            new PayPalAdapter(new PayPalGateway());
```

```
        PaymentProcessor stripeProcessor =  
            new StripeAdapter(new StripeGateway());
```

```
        payPalProcessor.processPayment(5000);  
        stripeProcessor.processPayment(7500);  
    }  
}
```

Output

Processing payment of Rs.5000.0 through PayPal.

Processing payment of Rs.7500.0 through Stripe.

Result

The Adapter Design Pattern was successfully implemented. Different third-party payment gateways with incompatible interfaces were integrated into a common payment processing interface using adapter classes, enabling seamless payment processing through a unified interface.

Exercise 5: Implementing the Decorator Pattern

Aim

To implement the Decorator Design Pattern in Java for a notification system that supports multiple notification channels such as Email, SMS, and Slack.

Objective

The Decorator Pattern allows behavior to be added to individual objects dynamically without modifying their class. It provides a flexible alternative to subclassing for extending functionality.

Algorithm

1. Create an interface named Notifier with a method send().
2. Create a concrete class EmailNotifier that implements the Notifier interface.
3. Create an abstract class NotifierDecorator that implements Notifier and contains a reference to a Notifier object.
4. Create concrete decorator classes:
 - SMSNotifierDecorator
 - SlackNotifierDecorator
5. Override the send() method in each decorator to add additional notification functionality.
6. Create a test class to demonstrate sending notifications through multiple channels by combining decorators.

Program

Notifier.java

```
public interface Notifier {  
    void send(String message);  
}
```

EmailNotifier.java

```
public class EmailNotifier implements Notifier {  
  
    @Override  
    public void send(String message) {  
        System.out.println("Sending Email: " + message);  
    }  
}
```

NotifierDecorator.java

```
public abstract class NotifierDecorator implements Notifier {  
  
    protected Notifier notifier;  
  
    public NotifierDecorator(Notifier notifier) {  
        this.notifier = notifier;  
    }  
  
    @Override  
    public void send(String message) {  
        notifier.send(message);  
    }  
}
```

SMSNotifierDecorator.java

```
public class SMSNotifierDecorator extends NotifierDecorator {
```

```

public SMSNotifierDecorator(Notifier notifier) {
    super(notifier);
}

@Override
public void send(String message) {
    super.send(message);
    sendSMS(message);
}

private void sendSMS(String message) {
    System.out.println("Sending SMS: " + message);
}
}

SlackNotifierDecorator.java
public class SlackNotifierDecorator extends NotifierDecorator {

    public SlackNotifierDecorator(Notifier notifier) {
        super(notifier);
    }

    @Override
    public void send(String message) {
        super.send(message);
        sendSlack(message);
    }

    private void sendSlack(String message) {
        System.out.println("Sending Slack Notification: " + message);
    }
}

DecoratorPatternTest.java
public class DecoratorPatternTest {

    public static void main(String[] args) {

        Notifier notifier = new EmailNotifier();

        System.out.println("Email Notification:");
        notifier.send("System Update Available");

        System.out.println("\nEmail + SMS Notification:");
        Notifier smsNotifier =

```

```

        new SMSNotifierDecorator(new EmailNotifier());
smsNotifier.send("System Update Available");

System.out.println("\nEmail + SMS + Slack Notification:");
Notifier multiChannelNotifier =
    new SlackNotifierDecorator(
        new SMSNotifierDecorator(
            new EmailNotifier()));

multiChannelNotifier.send("System Update Available");
}
}

```

Output

Email Notification:

Sending Email: System Update Available

Email + SMS Notification:

Sending Email: System Update Available

Sending SMS: System Update Available

Email + SMS + Slack Notification:

Sending Email: System Update Available

Sending SMS: System Update Available

Sending Slack Notification: System Update Available

Result

The Decorator Design Pattern was successfully implemented. Additional notification channels such as SMS and Slack were dynamically added to the basic Email notification without modifying the existing classes, demonstrating the flexibility and extensibility of the Decorator Pattern.

Exercise 6: Implementing the Proxy Pattern

Aim

To implement the Proxy Design Pattern in Java for an image viewer application that loads images from a remote server using lazy initialization and caching.

Objective

The Proxy Pattern provides a placeholder or surrogate for another object to control access to it. In this example, a proxy object delays the creation of a real image object until it is needed, thereby improving performance and reducing resource usage.

Algorithm

1. Create an interface named Image with a method display().
2. Create a class RealImage that implements the Image interface.
3. Simulate loading an image from a remote server in the constructor of RealImage.
4. Create a class ProxyImage that also implements the Image interface.
5. Store the image filename in ProxyImage.

6. Implement lazy initialization by creating the ReallImage object only when display() is called for the first time.
7. Reuse the same ReallImage object for subsequent requests to demonstrate caching.
8. Create a test class to verify the behavior of the proxy object.

Program

Image.java

```
public interface Image {  
    void display();  
}
```

ReallImage.java

```
public class ReallImage implements Image {  
  
    private String fileName;  
  
    public ReallImage(String fileName) {  
        this.fileName = fileName;  
        loadFromServer();  
    }  
  
    private void loadFromServer() {  
        System.out.println("Loading image from remote server: " + fileName);  
    }  
  
    @Override  
    public void display() {  
        System.out.println("Displaying image: " + fileName);  
    }  
}
```

ProxylImage.java

```
public class ProxylImage implements Image {  
  
    private String fileName;  
    private ReallImage reallImage;  
  
    public ProxylImage(String fileName) {  
        this.fileName = fileName;  
    }  
  
    @Override  
    public void display() {  
  
        if (reallImage == null) {  
            reallImage = new ReallImage(fileName);  
        }  
    }  
}
```

```

        reallImage.display();
    }
}
ProxyPatternTest.java
public class ProxyPatternTest {

    public static void main(String[] args) {

        Image image = new ProxyImage("sample.jpg");

        System.out.println("First Display:");
        image.display();

        System.out.println("\nSecond Display:");
        image.display();
    }
}

```

Output

First Display:

Loading image from remote server: sample.jpg

Displaying image: sample.jpg

Second Display:

Displaying image: sample.jpg

Explanation

- During the first call to display(), the ProxyImage creates a ReallImage object and loads the image from the remote server.
- During subsequent calls, the already-created ReallImage object is reused.
- This demonstrates both **lazy initialization** and **caching**, improving performance by avoiding repeated loading of the same image.

Result

The Proxy Design Pattern was successfully implemented. The ProxyImage class controlled access to the ReallImage object by delaying image loading until required and reusing the loaded image for future requests, thereby achieving lazy initialization and caching.

Exercise 7: Implementing the Observer Pattern

Aim

To implement the Observer Design Pattern in Java for a stock market monitoring application where multiple clients are notified whenever stock prices change.

Objective

The Observer Pattern defines a one-to-many dependency between objects so that when one object (subject) changes its state, all its dependent objects (observers) are notified

automatically. This pattern is commonly used in event handling systems, notification services, and real-time monitoring applications.

Algorithm

1. Create an interface named Stock with methods to register, deregister, and notify observers.
2. Create an interface named Observer with an update() method.
3. Implement a concrete subject class StockMarket that maintains a list of observers.
4. Implement methods to add, remove, and notify observers.
5. Create concrete observer classes MobileApp and WebApp that implement the Observer interface.
6. When the stock price changes, update the stock market and notify all registered observers.
7. Create a test class to demonstrate observer registration and notification.

Program

Observer.java

```
public interface Observer {  
    void update(String stockName, double stockPrice);  
}
```

Stock.java

```
public interface Stock {  
  
    void registerObserver(Observer observer);  
  
    void removeObserver(Observer observer);  
  
    void notifyObservers();  
}
```

StockMarket.java

```
import java.util.ArrayList;  
import java.util.List;  
  
public class StockMarket implements Stock {  
  
    private List<Observer> observers;  
    private String stockName;  
    private double stockPrice;  
  
    public StockMarket() {  
        observers = new ArrayList<>();  
    }  
  
    @Override  
    public void registerObserver(Observer observer) {  
        observers.add(observer);  
    }  
}
```

```
}
```

```
@Override  
public void removeObserver(Observer observer) {  
    observers.remove(observer);  
}
```

```
@Override  
public void notifyObservers() {  
    for (Observer observer : observers) {  
        observer.update(stockName, stockPrice);  
    }  
}
```

```
public void setStockData(String stockName, double stockPrice) {  
    this.stockName = stockName;  
    this.stockPrice = stockPrice;  
    notifyObservers();  
}  
}
```

MobileApp.java

```
public class MobileApp implements Observer {  
  
    private String userName;  
  
    public MobileApp(String userName) {  
        this.userName = userName;  
    }  
  
    @Override  
    public void update(String stockName, double stockPrice) {  
        System.out.println("Mobile App [" + userName +  
            "]" - " + stockName +  
            " price updated to Rs." + stockPrice);  
    }  
}
```

WebApp.java

```
public class WebApp implements Observer {  
  
    private String userName;  
  
    public WebApp(String userName) {  
        this.userName = userName;  
    }  
}
```

```

@Override
public void update(String stockName, double stockPrice) {
    System.out.println("Web App [" + userName +
        "]" - " + stockName +
        " price updated to Rs." + stockPrice);
}
}

```

ObserverPatternTest.java

```

public class ObserverPatternTest {

    public static void main(String[] args) {

        StockMarket stockMarket = new StockMarket();

        Observer mobileUser = new MobileApp("John");
        Observer webUser = new WebApp("Alice");

        stockMarket.registerObserver(mobileUser);
        stockMarket.registerObserver(webUser);

        System.out.println("Updating Stock Price...\n");

        stockMarket.setStockData("TCS", 3850.50);

        System.out.println("\nRemoving Web User...\n");

        stockMarket.removeObserver(webUser);

        stockMarket.setStockData("Infosys", 1625.75);
    }
}

```

Output

Updating Stock Price...

Mobile App [John] - TCS price updated to Rs.3850.5

Web App [Alice] - TCS price updated to Rs.3850.5

Removing Web User...

Mobile App [John] - Infosys price updated to Rs.1625.75

Explanation

- StockMarket acts as the Subject and maintains a list of observers.
- MobileApp and WebApp act as Observers.

- Whenever the stock price changes, StockMarket notifies all registered observers.
- If an observer is removed, it no longer receives updates.

Result

The Observer Design Pattern was successfully implemented. The StockMarket object notified all registered observers whenever stock prices changed, demonstrating the one-to-many relationship between the subject and its observers.

Exercise 8: Implementing the Strategy Pattern

Aim

To implement the Strategy Design Pattern in Java for a payment system that supports multiple payment methods such as Credit Card and PayPal.

Objective

The Strategy Pattern defines a family of algorithms, encapsulates each one, and makes them interchangeable. It allows the selection of a specific algorithm or behavior at runtime without modifying the client code.

Algorithm

1. Create an interface named PaymentStrategy with a method pay().
2. Create concrete strategy classes:
 - CreditCardPayment
 - PayPalPayment
3. Implement the pay() method in each strategy class.
4. Create a context class named PaymentContext.
5. Store a reference to a PaymentStrategy object in the context class.
6. Provide a method to set the payment strategy dynamically.
7. Execute the selected payment strategy through the context class.
8. Create a test class to demonstrate switching between different payment strategies at runtime.

Program

PaymentStrategy.java

```
public interface PaymentStrategy {
    void pay(double amount);
}
```

CreditCardPayment.java

```
public class CreditCardPayment implements PaymentStrategy {

    private String cardNumber;

    public CreditCardPayment(String cardNumber) {
        this.cardNumber = cardNumber;
    }

    @Override
    public void pay(double amount) {
        System.out.println("Paid Rs." + amount +
```

```

        " using Credit Card: " + cardNumber);
    }
}

```

PayPalPayment.java

```

public class PayPalPayment implements PaymentStrategy {

```

```

    private String email;

```

```

    public PayPalPayment(String email) {
        this.email = email;
    }

```

```

    @Override

```

```

    public void pay(double amount) {
        System.out.println("Paid Rs." + amount +
            " using PayPal Account: " + email);
    }
}

```

PaymentContext.java

```

public class PaymentContext {

```

```

    private PaymentStrategy paymentStrategy;

```

```

    public void setPaymentStrategy(PaymentStrategy paymentStrategy) {
        this.paymentStrategy = paymentStrategy;
    }

```

```

    public void executePayment(double amount) {
        if (paymentStrategy != null) {
            paymentStrategy.pay(amount);
        } else {
            System.out.println("No payment strategy selected.");
        }
    }
}

```

StrategyPatternTest.java

```

public class StrategyPatternTest {

```

```

    public static void main(String[] args) {

```

```

        PaymentContext context = new PaymentContext();

```

```

        // Pay using Credit Card
        context.setPaymentStrategy(

```

```

        new CreditCardPayment("1234-5678-9012-3456"));
context.executePayment(5000);

// Pay using PayPal
context.setPaymentStrategy(
    new PayPalPayment("user@example.com"));
context.executePayment(2500);
    }
}

```

Output

Paid Rs.5000.0 using Credit Card: 1234-5678-9012-3456

Paid Rs.2500.0 using PayPal Account: user@example.com

Explanation

- PaymentStrategy defines a common interface for all payment methods.
- CreditCardPayment and PayPalPayment provide different implementations of the payment algorithm.
- PaymentContext acts as the context and maintains a reference to the current strategy.
- The payment method can be changed dynamically at runtime without modifying the context class.

Result

The Strategy Design Pattern was successfully implemented. Different payment methods were encapsulated as separate strategies and selected dynamically at runtime through the PaymentContext class, demonstrating flexibility and maintainability in the payment system.

Exercise 9: Implementing the Command Pattern

Aim

To implement the Command Design Pattern in Java for a home automation system that can issue commands to turn devices on and off.

Objective

The Command Pattern encapsulates a request as an object, thereby allowing parameterization of clients with different requests, queuing of requests, and support for undoable operations. This pattern decouples the sender of a request from the object that performs the action.

Algorithm

1. Create an interface named Command with a method execute().
2. Create a receiver class Light with methods turnOn() and turnOff().
3. Create concrete command classes:
 - LightOnCommand
 - LightOffCommand
4. Each command class should implement the Command interface and invoke the appropriate method on the receiver.
5. Create an invoker class RemoteControl that stores a command object.
6. Provide methods to set and execute commands in the invoker.

7. Create a test class to demonstrate controlling the light using different commands.

Program

Command.java

```
public interface Command {  
    void execute();  
}
```

Light.java

```
public class Light {  
  
    public void turnOn() {  
        System.out.println("Light is ON");  
    }  
  
    public void turnOff() {  
        System.out.println("Light is OFF");  
    }  
}
```

LightOnCommand.java

```
public class LightOnCommand implements Command {  
  
    private Light light;  
  
    public LightOnCommand(Light light) {  
        this.light = light;  
    }  
  
    @Override  
    public void execute() {  
        light.turnOn();  
    }  
}
```

LightOffCommand.java

```
public class LightOffCommand implements Command {  
  
    private Light light;  
  
    public LightOffCommand(Light light) {  
        this.light = light;  
    }  
  
    @Override  
    public void execute() {  
        light.turnOff();  
    }  
}
```

```
}
```

RemoteControl.java

```
public class RemoteControl {  
  
    private Command command;  
  
    public void setCommand(Command command) {  
        this.command = command;  
    }  
  
    public void pressButton() {  
        command.execute();  
    }  
}
```

CommandPatternTest.java

```
public class CommandPatternTest {  
  
    public static void main(String[] args) {  
  
        // Receiver  
        Light light = new Light();  
  
        // Commands  
        Command lightOn = new LightOnCommand(light);  
        Command lightOff = new LightOffCommand(light);  
  
        // Invoker  
        RemoteControl remote = new RemoteControl();  
  
        System.out.println("Turning Light ON:");  
        remote.setCommand(lightOn);  
        remote.pressButton();  
  
        System.out.println("\nTurning Light OFF:");  
        remote.setCommand(lightOff);  
        remote.pressButton();  
    }  
}
```

Output

Turning Light ON:
Light is ON

Turning Light OFF:
Light is OFF

Explanation

- Command defines the interface for executing operations.
- Light acts as the Receiver that performs the actual actions.
- LightOnCommand and LightOffCommand encapsulate requests as objects.
- RemoteControl acts as the Invoker that triggers commands without knowing their implementation details.
- The client can dynamically assign different commands to the remote control.

Result

The Command Design Pattern was successfully implemented. Commands for turning a light on and off were encapsulated into separate command objects and executed through a remote control, demonstrating loose coupling between the invoker and receiver.

Exercise 10: Implementing the MVC Pattern

Aim

To implement the Model-View-Controller (MVC) Design Pattern in Java for a student record management system.

Objective

The MVC Pattern separates an application into three interconnected components: Model, View, and Controller. This separation helps organize code, improve maintainability, and support independent development of user interfaces and business logic.

Algorithm

1. Create a Model class named Student with attributes such as name, id, and grade.
2. Create a View class named StudentView with a method to display student details.
3. Create a Controller class named StudentController.
4. Store references to both the Student model and the StudentView view in the controller.
5. Provide methods in the controller to update and retrieve student details.
6. Use the controller to update the model and instruct the view to display the data.
7. Create a main class to demonstrate the interaction between the Model, View, and Controller.

Program

Student.java (Model)

```
public class Student {  
  
    private String name;  
    private String id;  
    private String grade;  
  
    public Student(String name, String id, String grade) {  
        this.name = name;  
        this.id = id;  
        this.grade = grade;  
    }  
}
```

```

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getId() {
        return id;
    }

    public void setId(String id) {
        this.id = id;
    }

    public String getGrade() {
        return grade;
    }

    public void setGrade(String grade) {
        this.grade = grade;
    }
}

```

StudentView.java (View)

```

public class StudentView {

    public void displayStudentDetails(String name,
                                     String id,
                                     String grade) {

        System.out.println("Student Details");
        System.out.println("-----");
        System.out.println("Name : " + name);
        System.out.println("ID   : " + id);
        System.out.println("Grade : " + grade);
    }
}

```

StudentController.java (Controller)

```

public class StudentController {

    private Student model;
    private StudentView view;
}

```

```

public StudentController(Student model,
                        StudentView view) {
    this.model = model;
    this.view = view;
}

public void setStudentName(String name) {
    model.setName(name);
}

public String getStudentName() {
    return model.getName();
}

public void setStudentId(String id) {
    model.setId(id);
}

public String getStudentId() {
    return model.getId();
}

public void setStudentGrade(String grade) {
    model.setGrade(grade);
}

public String getStudentGrade() {
    return model.getGrade();
}

public void updateView() {
    view.displayStudentDetails(
        model.getName(),
        model.getId(),
        model.getGrade()
    );
}
}

```

MVCPatternTest.java

```

public class MVCPatternTest {

    public static void main(String[] args) {

        // Create Model
    }
}

```

```

Student student =
    new Student("John", "S101", "A");

// Create View
StudentView view = new StudentView();

// Create Controller
StudentController controller =
    new StudentController(student, view);

// Display Initial Details
System.out.println("Initial Student Details:");
controller.updateView();

// Update Student Information
controller.setStudentName("Alice");
controller.setStudentGrade("A+");

System.out.println("\nUpdated Student Details:");
controller.updateView();
}
}

```

Output

Initial Student Details:
Student Details

Name : John
ID : S101
Grade : A

Updated Student Details:
Student Details

Name : Alice
ID : S101
Grade : A+

Explanation

- **Model (Student)** stores the student data.
- **View (StudentView)** is responsible for displaying the student information.
- **Controller (StudentController)** acts as an intermediary between the Model and the View.
- Any changes made through the Controller are reflected in the Model and displayed through the View.
- This separation improves code organization and maintainability.

Result

The MVC Design Pattern was successfully implemented. The StudentController managed communication between the Student model and the StudentView, demonstrating clear separation of concerns and efficient handling of student records.

Exercise 11: Implementing Dependency Injection

Aim

To implement Dependency Injection in Java using constructor injection for managing dependencies between a service class and a repository class.

Objective

Dependency Injection (DI) is a design pattern that allows objects to receive their dependencies from external sources rather than creating them internally. This promotes loose coupling, improves maintainability, and makes applications easier to test.

Algorithm

1. Create an interface named CustomerRepository with a method findCustomerById().
2. Create a concrete class CustomerRepositoryImpl that implements the CustomerRepository interface.
3. Create a service class named CustomerService.
4. Declare a dependency on CustomerRepository in CustomerService.
5. Use constructor injection to inject the repository dependency into the service class.
6. Create a main class to instantiate the repository and inject it into the service.
7. Use the service class to retrieve customer information.

Program

CustomerRepository.java

```
public interface CustomerRepository {  
    String findCustomerById(int id);  
}
```

CustomerRepositoryImpl.java

```
public class CustomerRepositoryImpl implements CustomerRepository {
```

```
    @Override  
    public String findCustomerById(int id) {  
        return "Customer ID: " + id +  
            ", Name: John Smith";  
    }  
}
```

CustomerService.java

```
public class CustomerService {  
  
    private CustomerRepository customerRepository;  
  
    // Constructor Injection  
    public CustomerService(CustomerRepository customerRepository) {  
        this.customerRepository = customerRepository;  
    }  
}
```

```

    }

    public String getCustomerDetails(int id) {
        return customerRepository.findCustomerById(id);
    }
}

```

DependencyInjectionTest.java

```

public class DependencyInjectionTest {

    public static void main(String[] args) {

        // Create Repository
        CustomerRepository repository =
            new CustomerRepositoryImpl();

        // Inject Repository into Service
        CustomerService service =
            new CustomerService(repository);

        // Use Service
        String customer =
            service.getCustomerDetails(101);

        System.out.println(customer);
    }
}

```

Output

Customer ID: 101, Name: John Smith

Explanation

- CustomerRepository defines the contract for accessing customer data.
- CustomerRepositoryImpl provides the actual implementation of the repository.
- CustomerService depends on CustomerRepository but does not create it directly.
- The dependency is supplied through the constructor (Constructor Injection).
- This approach reduces coupling between classes and makes the application more flexible and testable.

Advantages of Dependency Injection

1. Promotes loose coupling between components.
2. Improves code maintainability.
3. Simplifies unit testing by allowing mock dependencies.
4. Enhances code reusability and flexibility.

Result

The Dependency Injection pattern was successfully implemented using constructor injection. The CustomerService received its dependency (CustomerRepository) externally, demonstrating loose coupling and improved maintainability of the application.